# GigBid: why a correct bid decision is not a completed proposal

Raahim / AI-4016 / November 2026

GigBid is a freelancer copilot that finds jobs, checks risk and portfolio fit, proposes a price, drafts a grounded application, and prepares client replies. The practical constraint is that the freelancer remains responsible for every external action. A convincing paragraph is insufficient: the system must explain its evidence, preserve edits across restarts, and avoid spending Connects or sending messages without approval. I implemented a Next.js workspace, FastAPI service, LangGraph workflows, three MCP capability boundaries, and an independent student marketplace simulator. The strongest finding was that reliable early decisions did not imply reliable completion. In a controlled fifty-job comparison, the custom graph made 49 correct bid decisions but still encountered thirteen later failures.

## Data and what the outcomes mean

The application includes a reproducible historical-data pipeline using Asaniczka's Upwork Job Postings Dataset 2024, Kaggle version 1. The selected archive contains 53,058 rows; preprocessing produces 600 hybrid scenarios across AI/ML, full-stack development, and data analysis. Historical titles, descriptions, dates, and available posted budgets are distinguished from generated clients, skill labels, screening questions, inserted attacks, and outcomes. The uploader declares ODC Attribution terms. That declaration does not independently establish rights in every underlying job description, so raw historical text is excluded from the public repository. Attribution, input hashes, filtering, and limitations are recorded in the data card.

The architecture experiment uses a separate, fully synthetic fifty-job cohort. Three fictional personas provide clearly labelled coursework examples. Their case studies contain no established revenue uplift, years of professional experience, or client testimonials. Real BGE embeddings, Qdrant searches, MCP reads, and model calls operate on these sources. This gives reproducible execution without presenting invented work as a student's employment history. The simulator's rule client produces transparent eligibility and response outcomes. A simulated reply is a benchmark observation, not evidence that a real client would hire the freelancer. Compatibility with an unseen grading simulator is also unverified.

## Approval, persistence, and capability design

The API persists accounts, jobs, evidence revisions, runs, and approval records in Postgres. Redis supports workers and shared inference gates; Qdrant stores tenant-filtered evidence references. The bid graph fans out analysis, risk assessment, and portfolio matching before joining their typed results. Pricing, writing, sentence verification, bounded Critic revisions, and compliance checks lead to a human interrupt. The conversation graph uses a durable thread per conversation and the same review principle. Separate stores hold current workflow state, execution events, and explicitly promoted long-term memory.

Approval binds a particular preview, version, digest, tool, payload, and idempotency key. Editing the draft invalidates that approval. The executor rechecks source freshness, balances, and connection state before acting. The simulator atomically deduplicates proposal effects and Connects debits, while reconciliation

can recover an already observed effect after a worker interruption. This prevents a retry from becoming a second payment or submission. Scheduled discovery may queue bounded analysis, but it never authorizes a proposal. Memory similarly requires a promotion rule: repeated confirmed edits can establish style preferences, while outcome observations must come from trusted provider events.

Portfolio, marketplace, and workspace MCP servers expose different capabilities. Tenant and run identities come from the host rather than model-selected tool arguments. Read access cannot manufacture write permission. Tool descriptions and structured Pydantic handoffs make contracts inspectable, but they do not guarantee the model will produce a valid or useful result. Control tests therefore establish approval and recovery behavior separately from the research measurements of model quality.

## Retrieval was selected by measurement

I compared dense retrieval V0, requirement decomposition V1, hybrid fusion V2, hybrid reranking V3, and corrective retrieval V8. The controlled set contains sixty answerable tasks and six unrelated tasks. V0 and V3 both achieved Recall@5 of 1.000, so the hypothesis that reranking would improve recall was not supported on this easy cohort. V3 improved MRR@5 from approximately 0.987 to 1.000. Requirement decomposition performed worse: generic requirements could dominate fusion and reduce recall to 0.725. The declared recall, ordering, and latency selection rule therefore selected V3.

Corrective V8 abstained on all six unrelated queries, whereas the other variants returned evidence for all six. That improvement came at a cost: V8 p95 retrieval latency was 10.715 seconds against V3's 3.342 seconds. The result supports a specific deployment trade-off, not a general claim that hybrid retrieval always wins. The corpus is small and templated, relevance gold follows project identity, and real-job relevance has not been independently human-labelled. The saved rankings and per-persona metrics make those limits visible.

## Models, architecture, and the cost of coordination

The required open-weight comparison ran thirty jobs each with Qwen3.5 4B and Qwen3 8B locally. Both configurations produced no simulator submissions. Grounding failures dominated, and diagnostic inspection exposed a contract problem: future plans could be treated as unsupported past experience. The revised contract explicitly distinguishes non-factual plans while retaining checks on factual achievements. Those fixes do not erase the negative baseline. Because prompts, contracts, and models changed afterward, the later hosted study cannot isolate a model-size improvement over those local results.

The optional hosted comparison ran the same fifty jobs through ReAct, the actual langgraph-supervisor package, and the production custom graph. Gemini 3.1 and 3.5 Flash-Lite assignments were blocked by job and balanced within persona, so all three architectures saw the same assigned model for each job. The protocol used a 20,000-token thread budget, thirty native-graph steps, forty custom-graph steps, and at most two Critic revisions. Source snapshots and all 150 observations were retained. Failures and drafts from unsuccessful runs were not silently replaced with successful reruns.

ReAct completed 41/50 runs without errors, supervisor 24/50, and custom graph 37/50. Intermediate correct decisions were 36, 24, and 49 respectively. When requiring an error-free run as well as a correct decision, the counts became 36, 23, and 36. This distinction matters: the custom graph's 98% decision accuracy is not 98% end-to-end success. Simulated replies per assigned job were 35/50, 18/50, and

29/50. The supervisor's additional coordination and context were difficult to sustain under the declared budget; this protocol does not prove supervisors are inherently inferior. Safety changes the interpretation: ReAct rejected only one of six scam controls, supervisor five, and the custom graph all six. Five ReAct scam bids and one supervisor scam bid reached the isolated simulator. Higher completion alone therefore does not justify replacing the custom graph. No actual user approval or real client submission was fabricated.

Across the comparison, 48 runs recorded errors: 39 budget stops, four timeouts, three provider or structured-output failures, and two other observed stops. Recorded p95 latencies were approximately 139, 186, and 179 seconds, exceeding the sixty-second analysis target. These times include throttling within attempts but do not fully include daily-quota pauses between attempts. Successful-call usage totals 1,368,464 measured tokens; 58 native calls have unknown usage. Actual API billing was not independently verified and is reported as unknown. Free-tier availability is not a measured USD cost or an unlimited allowance.

## Human evidence, quality, and the extension

The independent Nvidia Nemotron judge was calibrated against thirty student-supplied evidence-and-claim labels. Claim-label agreement was 60%, with Cohen's kappa 0.403. Exact quality-score agreement was 46.7%, with quadratic weighted kappa 0.681 on the five-point scale. Consequently, automated unsupported-claim and quality scores are proxy measurements rather than ground truth. Output-protocol errors, including invented claim indices, were rejected and retained as diagnostic attempts. Schema repairs preserved the scoring rubric rather than tuning it to match the human answers.

Two user-supplied classmate responses rated twelve anonymized pairs of raw local-model proposals. Qwen 4B received thirteen votes and Qwen 8B eleven; reviewers agreed on eleven pairs, kappa 0.833. This small convenience panel does not establish a meaningful model winner. Twenty-four ratings represent twelve repeated jobs, not twenty-four independent examples. Reviewer identities and independence were not independently verified, and the second response was reconstructed from the user's stated final-choice difference. These observations concern the older local candidates, not the Gemini architecture study.

For the X2 extension, the custom-graph jobs were assigned before execution to evidence-first or problem-first openings, twenty-five per arm. Intention-to-treat replies were 12/25 and 17/25. The evidence-minus-problem difference was minus twenty percentage points, but the two-sided Fisher p-value was 0.252. A conservative 95% Hoeffding interval spanned minus 79.2 to plus 39.2 points. The experiment therefore does not establish a strategy benefit. The pooled test ignores assignment strata; per-persona counts are retained, and all outcomes come from the rule simulator.

A supplemental paired diagnostic ran nine identical jobs per retrieval variant using the same hosted open profile. V0 produced two replies and V3 one; failures stayed in the denominator. V3's lower latency included early stops, so it does not establish a speed gain. Four changed verifier checkpoint pairs were independently judged: unsupported factual claims were 2/22 before and 1/13 after, with unchanged mean quality of 3.75. Nine unchanged control pairs received inconsistent factual labels, exposing judge noise. These small retrospective samples support caution, not a broad grounding-improvement claim.

## What remains defensible

The forty-task tool-design study adds a narrower result: fine, coarse, and rewritten tools all achieved perfect selection and argument accuracy in 120 GPT-OSS 120B calls. Fine tools used about 618 tokens per task against 989 for coarse tools. The accuracy ceiling prevents a strong behavior claim. Together, the studies support reducing context and verification overhead before adding coordination. Durable approval and factual grounding remain necessary even when they expose failures. Temporary laptop-backed HTTPS hosting, an authenticated Portfolio MCP endpoint, commit-pinned GitHub README/source ingestion and public metadata-only trace cases are verified. Real Upwork OAuth and live job reads were subsequently verified; provider-preview writes remain fixture-tested, and final submissions and direct messages stay disabled. A recorded API crash preserved approval and yielded one simulator receipt. The demo video, GIF, installable MCP command and 214 backend/17 browser checks are available. The public app is off; always-on hosting and hidden-test performance remain unverified. Codex assisted implementation and analysis; human ratings came from the student and reported classmates. Results files and reproducible scripts distinguish these evidence sources.

## Evidence and reproduction

Repository: https://github.com/raahim-ibrar/GigBidA2. Numerical evidence is in evals/results/student-data-manifest.json, student-retrieval/results.json, student-generation-v3/progress.json, student-tool-study-v3/results.json, gemini-orchestration-v1/analysis.json, judge-calibration-v4/agreement.json, judge-proposals-final/results.json, human-panel-v1/results.json, rag-end-to-end-v1/analysis.json, and verification-changed-v1/analysis.json. Run python -m evals.analyse_final_research and python -m evals.analyse_proposal_quality to regenerate the final analysis without inference. See DATA_CARD.md, VERIFICATION.md, and AI_USAGE.md for provenance, control checks, and assistance disclosure.